

# 第1章 计算机系统漫游

本章从整体视角介绍计算机系统的组成与运行机制，建立"系统是硬件和软件协同工作的集合"的核心认知。

## 1. 程序的生命周期

C语言源程序（文本文件）经过四个阶段转换为可执行文件：

- **预处理 (cpp)**：处理 `#include`、`#define` 等指令，生成 `.i` 文本文件
- **编译 (cc1)**：将C代码翻译为汇编代码，生成 `.s` 文本文件
- **汇编 (as)**：将汇编代码翻译为机器指令，生成可重定位目标文件 `.o`（二进制文件）
- **链接 (ld)**：合并目标文件和库文件，生成最终可执行文件（二进制文件）

## 2. 系统硬件组成

- **CPU**：包含程序计数器(PC)、寄存器文件、算术逻辑单元(ALU)，通过总线与其他部件通信
- **主存**：临时存储程序和数据，由 DRAM 芯片组成，按**字节**寻址
- **I/O设备**：磁盘、键盘、显示器、网络适配器等，通过I/O总线与系统连接
- **总线**：系统总线、内存总线、I/O总线，负责在部件间传输数据

## 3. 存储层次结构

遵循"速度越快、容量越小、单位成本越高"的规律，从上到下依次为：

CPU寄存器 → L1高速缓存 → L2高速缓存 → L3高速缓存 → 主存 → 磁盘 → 远程存储

利用程序的时间局部性和空间局部性，上层存储作为下层存储的高速缓存，大幅提升访问效率。

## 4. 操作系统的核心抽象

操作系统作为应用程序与硬件之间的中间层，提供三个关键抽象：

- **文件**：对所有I/O设备的统一抽象
- **虚拟内存**：对主存和磁盘的抽象，为每个进程提供独立的虚拟地址空间
- **进程**：对处理器、主存和I/O设备的抽象，是运行中程序的实例
- **虚拟机**：对整个计算机系统的抽象，支持运行不同操作系统的程序

## 5. 进程的虚拟地址空间（32位/64位通用结构）

从低地址到高地址依次为：

1. 代码和数据段：程序指令和全局变量，大小在程序启动时固定
2. 堆：运行时动态分配的内存（`malloc / free`），可动态扩展收缩
3. 共享库：存放C标准库、数学库等共享代码和数据
4. 用户栈：编译器用于实现函数调用，函数调用时增长、返回时收缩
5. 内核虚拟内存：操作系统保留区域，应用程序无法直接访问

## 6. 网络通信

网络本质上是一种特殊的I/O设备。客户端-服务器模型是最常见的网络交互模式：客户端发送请求，服务器接收请求、处理并返回响应。

## 7. 贯穿全书的重要主题

### (1) Amdahl定律

系统整体加速比取决于**被优化部分的占比和加速程度**：

$$[ S = \frac{1}{(1-\alpha) + \alpha/k} ]$$

其中  $S$  是系统整体加速比， $\alpha$  是被优化部分的时间占比， $k$  是该部分的加速比。极限情况（ $k \rightarrow \infty$ ）下，最大加速比为（ $S_{\infty} = \frac{1}{1-\alpha}$ ）。

核心结论：要显著提升系统整体性能，必须优化占比很大的部分。

### (2) 并发与并行

- **线程级并发**：通过多进程/多线程实现，多核处理器和超线程技术可真正并行执行多个线程
- **指令级并行**：通过**流水线**、**超标量技术**，一个时钟周期内执行多条指令
- **SIMD并行**：**单指令多数据**，一条指令同时对多个数据执行相同操作，常用于多媒体和科学计算

### (3) 抽象的重要性

不同层次的抽象隐藏底层实现复杂性，提升代码的可移植性和开发效率。例如指令集架构抽象了处理器硬件，C语言数据类型抽象了机器级数据表示。

---

## 第2章 信息的表示和处理

本章深入讲解计算机中信息的二进制编码方式，以及整数和浮点数的运算规则，是理解程序底层行为的基础。

# 1. 信息存储

- **基本单位：**字节（8位）。虚拟内存被视为一个巨大的字节数组，每个字节有唯一的虚拟地址。
- **字节顺序：**
  - **小端法：**低字节存放在低地址，高字节存放在高地址（Intel x86系列）
  - **大端法：**高字节存放在低地址，低字节存放在高地址（IBM、Oracle SPARC）
  - 跨平台数据传输、网络通信必须处理字节序差异
- **C语言数据大小：**

| 类型     | 32位系统 | 64位系统 |
|--------|-------|-------|
| char   | 1字节   | 1字节   |
| short  | 2字节   | 2字节   |
| int    | 4字节   | 4字节   |
| long   | 4字节   | 8字节   |
| 指针     | 4字节   | 8字节   |
| float  | 4字节   | 4字节   |
| double | 8字节   | 8字节   |

- **位级运算：**
  - 布尔运算：`&`（与）、`|`（或）、`~`（非）、`^`（异或），按位操作
  - 逻辑运算：`&&`（与）、`||`（或）、`!`（非），结果为0（假）或1（真），具有短路求值特性
  - 移位运算：
    - 左移 `<<`：丢弃高位，低位补0
    - 逻辑右移 `>>`：丢弃低位，高位补0（用于无符号数）
    - 算术右移 `>>`：丢弃低位，高位补符号位（用于有符号补码数）
- **进制转换方法：**
  - ① **十进制 → 其他进制**
    - 整数：除基取余 — 不断除以基数，记余数，商为0后从下往上读

- 小数：乘基取整 — 不断乘以基数，记整数部分，直到小数部分为0（或达到精度），从上往下读
- ② 其他进制 → 十进制：按权展开 — 各位数字 × 对应位权，求和
- ③ 二进制 ↔ 十六进制：分组法 — 从小数点向左右每4位一组，不足补0，每组对应1位十六进制

## 2. 整数表示

### (1) 无符号数编码

将位向量直接解释为二进制数：

$$[B2U_w(\vec{x}) = \sum_{i=0}^{w-1} x_i 2^i]$$

取值范围：( $0 \sim 2^w - 1$ )，每个数值有唯一的位表示。

### (2) 补码编码（有符号数标准表示）

最高位为符号位，权重为( $-2^{w-1}$ )，其余位为正权重：

$$[B2T_w(\vec{x}) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i]$$

取值范围：( $-2^{w-1} \sim 2^{w-1} - 1$ )，负数范围比正数大1（最小负数 TMin 没有对应的正数）。

### (3) 类型转换与扩展截断

- 有符号 ↔ 无符号转换：位模式不变，仅改变解释方式。负数转无符号数会变成很大的正数。
- 扩展：无符号数用零扩展（高位补0），补码数用符号扩展（高位补符号位），数值保持不变。
- 截断：丢弃高位，无符号数结果为( $x \bmod 2^k$ )，补码数先取模再转换为有符号数，可能改变数值。

### (4) C语言隐式转换陷阱

当表达式中同时出现有符号和无符号数时，有符号数会隐式转换为无符号数，导致比较、运算出现非直观结果。例如  $-1 < 0U$  会被解释为  $4294967295U < 0U$ ，结果为假。

## 3. 整数运算

计算机整数运算本质是模运算，有限字长导致溢出，与数学上的整数运算有本质区别。

### (1) 加法

- 无符号加法：模( $2^w$ )运算，溢出时结果为( $x+y-2^w$ )。溢出检测：结果小于任一操作数。
- 补码加法：同样模( $2^w$ )，分三种情况：

- 正常：( $-2^{w-1} \leq x+y < 2^{w-1}$ )，结果为( $x+y$ )
  - 正溢出：( $x+y \geq 2^{w-1}$ )，结果为( $x+y-2^w$ ) (负数)
  - 负溢出：( $x+y < -2^{w-1}$ )，结果为( $x+y+2^w$ ) (正数)
- 溢出检测：两个正数相加得负，或两个负数相加得正。

## (2) 非与乘法

- **补码非**：TMin 的非是自身，其他数的非为  $-x$ 。位级实现： $\sim x + 1$ 。
- **乘法**：无符号和补码乘法的**低w位完全相同**，高w位不同。编译器会用移位和加减组合优化乘以常数的运算（例如  $x*14$  优化为  $(x \ll 3) + (x \ll 2) + (x \ll 1)$ ）。

## (3) 除法

除以2的幂用右移实现：

- 无符号数：逻辑右移，直接得到( $\lfloor x/2^k \rfloor$ )
- 补码数：算术右移，负数需先加**偏置量**( $2^{k-1}$ )再右移，实现**向零舍入**。

## (4) 安全提示

整数溢出是常见的程序漏洞和安全隐患（例如缓冲区溢出、内存分配错误），编写代码时必须检查运算结果是否溢出。

# 4. 浮点数表示

IEEE 754标准是现代计算机统一的浮点表示规范，通过科学计数法近似表示实数。

## (1) 表示形式

$$[V = (-1)^s \times M \times 2^E]$$

- **符号位s**：1位，0表示正，1表示负
- **尾数M**：有效数字，范围( $1 \leq M < 2$ ) (规格化) 或( $0 \leq M < 1$ ) (非规格化)
- **阶码E**：对浮点数加权，权重为2的E次幂

## (2) 格式分类

- **单精度float**：32位 (1位s + 8位阶码 + 23位尾数)
- **双精度double**：64位 (1位s + 11位阶码 + 52位尾数)

## (3) 数值类型

1. **规格化值**：阶码不全为0也不全为1

- ( $E = e - \text{Bias}$ ), 其中 $e$ 是阶码的无符号值,  $\text{Bias} = (2^{k-1} - 1)$  (单精度127, 双精度1023)
- ( $M = 1 + f$ ),  $f$ 是尾数的小数值 (隐含最高位1, 节省1位精度)

## 2. 非规格化值: 阶码全为0

- ( $E = 1 - \text{Bias}$ )
- ( $M = f$ ) (无隐含1)
- 用于表示0和非常接近0的数, 实现逐渐下溢

## 3. 特殊值: 阶码全为1

- 尾数全为0: 表示无穷大 ( $+\infty$ 、 $-\infty$ )
- 尾数非0: 表示NaN (不是一个数, 例如 $\sqrt{-1}$ 、 $0/0$ )

## (4) 舍入模式

默认采用**向偶数舍入** (中间值舍入到最低有效位为偶数), 避免统计偏差。其他舍入方式: 向零舍入、向下舍入、向上舍入。

## (5) 浮点运算特性

- 不满足**结合律**: 例如  $(3.14 + 1e20) - 1e20 = 0$ , 而  $3.14 + (1e20 - 1e20) = 3.14$
- 不满足**分配律**: 例如  $1e20 * (1e20 - 1e20) = 0$ , 而  $1e20 * 1e20 - 1e20 * 1e20 = \text{NaN}$
- 满足单调性:
  - **加法单调性**: 若  $(a \geq b)$ , 则对任意非NaN的 $x$ , 有  $(x + a \geq x + b)$
  - **乘法单调性**: 若  $(a \geq b)$ , 则对任意非NaN且  $(c \geq 0)$  的 $c$ , 有  $(a * c \geq b * c)$
- 不满足**精确相等**: 浮点舍入误差导致  $a == b$  不可靠, 应用  $|a - b| < \epsilon$  判断近似相等 ( $\epsilon$  取一个很小的正数, 如  $1e-9$ )

## (6) 类型转换

- $\text{int} \rightarrow \text{float}$ : 不会溢出, 但可能丢失精度 (int有32位精度, float只有23位尾数)
- $\text{int}/\text{float} \rightarrow \text{double}$ : 可精确表示, 因为double精度更高
- $\text{double} \rightarrow \text{float}$ : 可能溢出为无穷大, 也可能丢失精度
- $\text{float}/\text{double} \rightarrow \text{int}$ : 向零舍入, 值过大时会得到不确定结果

# 第3章 程序的机器级表示

## 3.1 历史观点

- **x86架构演进：**从Intel 8086（16位）→ 80386（32位IA-32）→ x86-64（64位）
- **x86-64优势：**
  - 扩展了寄存器数量（从8个到16个通用寄存器）
  - 每个寄存器宽度从32位增加到64位
  - 支持更大的虚拟地址空间：理论上2^64字节，实际使用48位（256TB）
    - **原因：**48位地址空间已足够大，简化硬件设计，减少页表层级
    - **规范地址：**高16位必须是第47位的符号扩展（全0或全1），形成两个独立的地址区间：
      - 用户空间：0x0000000000000000 ~ 0x00007FFFFFFFFFFFFF（低128TB）
      - 内核空间：0xFFFFF80000000000 ~ 0xFFFFFFFFFFFFFFFF（高128TB）
    - 非规范地址访问会触发段错误
- **指令集复杂性：**x86是CISC（复杂指令集计算机）架构，指令长度可变（1-15字节）
- **编译器角色：**将C语言程序转换为机器指令，负责寄存器分配、指令选择和优化

## 3.2 程序编码

### 3.2.1 机器级代码

- **两种抽象：**
  1. 指令集架构（ISA）：定义了处理器状态、指令格式和指令行为
  2. 虚拟地址：程序使用的逻辑地址，由硬件MMU依据OS维护的页表转换为物理地址
- **处理器可见状态：**
  - **程序计数器（PC）：**存放下一条指令的地址
  - **寄存器文件：**16个通用寄存器

| 寄存器  | 用途         | 保存约定 |
|------|------------|------|
| %rax | 返回值、乘除累加器  | 调用者  |
| %rbx | 基址寄存器      | 被调用者 |
| %rcx | 第4参数、循环计数器 | 调用者  |
| %rdx | 第3参数、乘除辅助  | 调用者  |
| %rsi | 第2参数、源指针   | 调用者  |
| %rdi | 第1参数、目的指针  | 调用者  |

|           |            |      |
|-----------|------------|------|
| %rbp      | 帧指针（可选）    | 被调用者 |
| %rsp      | 栈指针        | 被调用者 |
| %r8-%r9   | 第5/第6参数    | 调用者  |
| %r10-%r11 | 临时/系统调用返回值 | 调用者  |
| %r12-%r15 | 通用临时       | 被调用者 |

- **调用约定：**调用者保存（函数可随意改写）— %rax, %rcx, %rdx, %rsi, %rdi, %r8-%r11；被调用者保存（使用前须保存原值，返回前恢复）— %rbx, %rbp, %rsp, %r12-%r15
- **标志位寄存器：**保存算术/逻辑操作状态（ZF/CF/SF/OF）
- **向量寄存器：**存储浮点和整数向量数据

### 3.2.2 代码示例

- **编译过程：**C源文件 → 预处理 → 编译 → 汇编 → 链接 → 可执行文件
- **汇编代码特点：**
  - 使用助记符表示机器指令
  - 操作数：立即数、寄存器、内存地址三者之一
  - **内存操作数限制：**x86指令最多允许一个内存操作数，不能两个操作数同时为内存
  - 指令顺序与程序执行顺序一致
- **反汇编：**将机器代码转换回汇编代码，使用 `objdump -d`

### 3.2.3 关于格式的说

- **AT&T汇编格式：**本书使用的格式，特点是：
  - 源操作数在前，目的操作数在后
  - 寄存器名前加 % 前缀
  - 立即数前加 \$ 前缀
  - 内存引用使用 ( ) 表示，格式为 (%rsp)

## 3.3 数据格式

- **x86-64数据类型：**

| C声明 | Intel数据类型 | 汇编代码后缀 | 大小（字节） |
|-----|-----------|--------|--------|
|-----|-----------|--------|--------|



|        |     |   |   |
|--------|-----|---|---|
| char   | 字节  | b | 1 |
| short  | 字   | w | 2 |
| int    | 双字  | l | 4 |
| long   | 四字  | q | 8 |
| char * | 四字  | q | 8 |
| float  | 单精度 | s | 4 |
| double | 双精度 | l | 8 |

- **注意：** long double 类型在x86-64中通常是80位扩展精度，占用16字节内存

## 3.4 访问信息

### 3.4.1 操作数指示符

- **三种操作数类型：**
  1. **立即数：** 常量值，如 `$0x1F`
  2. **寄存器：** 寄存器内容，如 `%rax`
  3. **内存引用：** 根据有效地址访问内存，格式为 `Imm(Rb, Ri, S)`
    - 有效地址计算： $\text{Imm} + \text{Rb} + \text{Ri} * \text{S}$
    - Imm：偏移量（立即数）
    - Rb：基址寄存器
    - Ri：变址寄存器
    - S：比例因子（1, 2, 4, 8）

### 3.4.2 数据传送指令

- **基本传送指令：** mov 系列
  - `movb`：传送字节
  - `movw`：传送字
  - `movl`：传送双字（会将目标寄存器的高32位清零）
  - `movq`：传送四字
- **零扩展传送：** movz 系列，将较小的数据类型零扩展到较大的类型

- `movzbl` : 字节→双字
- `movzwl` : 字→双字
- `movzbq` : 字节→四字
- `movzwq` : 字→四字
- **符号扩展传送:** `movs` 系列, 将较小的数据类型符号扩展到较大的类型
  - `movsbq` : 字节→四字
  - `movswq` : 字→四字
  - `movslq` : 双字→四字
- **注意:** 没有 `movzql` 指令, 因为 `movl` 已经会将高32位清零

### 3.4.3 压入和弹出栈数据

- **栈操作:**
  - `pushq S` : 将四字压入栈
    - 栈指针 `%rsp` 减8
    - 将S的值写入 `(%rsp)`
  - `popq D` : 从栈弹出四字到D
    - 从 `(%rsp)` 读取值到D
    - 栈指针 `%rsp` 加8
- **栈的特点:**
  - 栈向下增长 (向低地址方向)
  - 栈顶元素的地址是 `%rsp` 指向的地址
  - 栈用于过程调用时保存返回地址、寄存器和局部变量

## 3.5 算术和逻辑操作

### 3.5.1 加载有效地址

- **leaq指令:** `leaq S, D`
  - 功能: 计算有效地址并存储到D
  - 用途: 不仅可以计算地址, 还可以进行简单的算术运算
  - 示例: `leaq 7(%rdx, %rdx, 4), %rax` → `%rax = 5 * %rdx + 7`

### 3.5.2 一元和二元操作

- **一元操作：**只有一个操作数，既是源又是目的
  - `inc D`：加1
  - `dec D`：减1
  - `neg D`：取负
  - `not D`：按位取反
- **二元操作：**两个操作数，第一个是源，第二个是目的
  - `add S, D`： $D = D + S$
  - `sub S, D`： $D = D - S$
  - `imul S, D`： $D = D * S$ （低64位）
  - `xor S, D`： $D = D \wedge S$
  - `or S, D`： $D = D | S$
  - `and S, D`： $D = D \& S$

### 3.5.3 移位操作

- **左移：**
  - `sal k, D`：算术左移，与逻辑左移相同
  - `shl k, D`：逻辑左移
- **右移：**
  - `sar k, D`：算术右移，符号扩展
  - `shr k, D`：逻辑右移，零扩展
- **移位量：**可以是立即数或 `%cl` 寄存器（只使用低6位）

### 3.5.4 特殊的算术操作

- **全64位乘法：**
  - `imulq S`：`%rax * S`，结果的低64位在 `%rax`，高64位在 `%rdx`
- **除法：**
  - `idivq S`：有符号除法，`%rdx:%rax` 除以 `S`
    - 商在 `%rax`，余数在 `%rdx`
  - `divq S`：无符号除法
- **注意：**除法前需要将被除数符号扩展到 `%rdx:%rax`

- 有符号：cqto 指令（将 %rax 符号扩展到 %rdx）
- 无符号：将 %rdx 清零

## 3.6 控制

### 3.6.1 条件码

- **主要条件码寄存器：**
  - CF：进位标志，无符号运算溢出时置1
  - ZF：零标志，结果为零时置1
  - SF：符号标志，结果为负时置1
  - OF：溢出标志，有符号运算溢出时置1
- **设置条件码的指令：**
  - 所有算术和逻辑操作都会设置条件码
  - `cmp S1, S2`：比较S2和S1，相当于 `S2 - S1` 但不修改目的寄存器
  - `test S1, S2`：按位与S1和S2，相当于 `S1 & S2` 但不修改目的寄存器
- **常见用法：**

| 用法       | 指令示例                          | 含义         | 标志位判断       |
|----------|-------------------------------|------------|-------------|
| 测试是否为零   | <code>test %rax, %rax</code>  | 检查%rax是否为0 | ZF=1表示为零    |
| 测试某位是否为1 | <code>test \$0x80, %al</code> | 检查%al的最高位  | ZF=0表示该位为1  |
| 测试是否为负数  | <code>test %rax, %rax</code>  | 检查%rax是否为负 | SF=1表示为负    |
| 测试是否为偶数  | <code>test \$1, %eax</code>   | 检查最低位      | ZF=1表示为偶数   |
| 测试多个标志位  | <code>test \$0x0F, %al</code> | 检查低4位是否有1  | ZF=1表示低4位全0 |

### 3.6.2 访问条件码

- **条件传送指令：**根据条件码的值传送数据
  - `cmove`：等于时传送（ZF=1）
  - `cmovne`：不等于时传送（ZF=0）
  - `cmovs`：负时传送（SF=1）
  - `cmovns`：非负时传送（SF=0）
  - `cmovg`：大于时传送（有符号）

- `cmovge`：大于等于时传送（有符号）
  - `cmovl`：小于时传送（有符号）
  - `cmovle`：小于等于时传送（有符号）
  - `cmova`：高于时传送（无符号）
  - `cmovae`：高于等于时传送（无符号）
  - `cmovb`：低于时传送（无符号）
  - `cmovbe`：低于等于时传送（无符号）
- **条件传送的优势**：避免分支预测错误带来的性能损失

### 3.6.3 跳转指令

- **无条件跳转**：`jmp Label`
- **条件跳转**：根据条件码的值跳转，后缀与条件传送相同
  - `je Label`：等于时跳转
  - `jne Label`：不等于时跳转
  - 等等...
- **跳转目标**：可以是直接地址（标签）或间接地址（寄存器或内存中的值）

### 3.6.4 跳转指令的编码

- **PC相对寻址**：跳转目标地址 = 当前PC值 + 偏移量
- **偏移量**：可以是1字节、2字节或4字节，有符号数
- **优势**：代码可以在内存中任意位置加载而不需要修改

### 3.6.5 用条件控制来实现条件分支

- **if-else语句的汇编实现**：

```
if (test-expr)
    then-statement
else
    else-statement
```

转换为：

```

    test test-expr
    jne else_label
    then-statement
    jmp end_label
else_label:
    else-statement
end_label:

```

### 3.6.6 用条件传送来实现条件分支

- 条件表达式的汇编实现：

```
v = test-expr ? then-expr : else-expr;
```

转换为：

```

movq else-expr, %rax
test test-expr
cmovne then-expr, %rax

```

- 条件传送的限制：
  - 两个表达式都必须是简单的计算
  - 不能有副作用
  - 计算代价不能太高

### 3.6.7 循环

- do-while循环：

```

do {
    body-statement
} while (test-expr);

```

转换为：

```
loop:
    body-statement
    test test-expr
    jne loop
```

- **while**循环:

```
while (test-expr)
    body-statement
```

转换为（跳转到中间）:

```
    jmp test
loop:
    body-statement
test:
    test test-expr
    jne loop
```

- **for**循环:

```
for (init-expr; test-expr; update-expr)
    body-statement
```

转换为:

```
    init-expr
    jmp test
loop:
    body-statement
    update-expr
test:
    test test-expr
    jne loop
```

### 3.6.8 switch语句

- **跳转表实现：**
  - 当case标签数量较多且范围较小时，使用 跳转表
  - 跳转表是一个数组，每个元素是对应case的代码地址
  - 根据switch表达式的值计算索引，跳转到对应的代码
- **优势：**时间复杂度 $O(1)$ ，比一系列if-else语句更快

## 3.7 过程

### 3.7.1 运行时栈

- **栈帧结构：**
  - 调用者栈帧：保存调用者的寄存器和局部变量
  - 被调用者栈帧：保存被调用者的参数、返回地址、寄存器和局部变量
- **栈指针：** `%rsp` 指向栈顶
- **帧指针：** `%rbp` 可选，用于指向栈帧底部，方便访问局部变量和参数

### 3.7.2 转移控制

- **call指令：** `call Label`
  - 将返回地址（call指令的下一条指令地址）压入栈
  - 跳转到Label
- **ret指令：**
  - 从栈弹出返回地址
  - 跳转到返回地址

### 3.7.3 数据传送

- **参数传递 (x86-64)：**
  - 前6个整数/指针参数通过寄存器传递：
    - `%rdi`：第1个参数
    - `%rsi`：第2个参数
    - `%rdx`：第3个参数
    - `%rcx`：第4个参数
    - `%r8`：第5个参数
    - `%r9`：第6个参数



- 超过6个的参数通过栈传递，从右向左压入
- **返回值**：通过 `%rax` 寄存器传递

### 3.7.4 寄存器使用惯例

- **调用者保存寄存器**： `%rax, %rcx, %rdx, %rsi, %rdi, %r8, %r9, %r10, %r11`
  - 被调用者可以修改这些寄存器而不需要保存
  - 如果调用者需要这些寄存器的值，必须在调用前保存
- **被调用者保存寄存器**： `%rbx, %rbp, %r12, %r13, %r14, %r15`
  - 被调用者在修改这些寄存器前必须保存
  - 返回前必须恢复这些寄存器的值

### 3.7.5 递归过程

- **递归调用的实现**：
  - 每次递归调用都会创建一个新的栈帧
  - 栈帧中保存当前调用的参数、局部变量和返回地址
  - 递归深度受限于栈的大小

## 3.8 数组的分配和访问

### 3.8.1 基本原则

- **数组元素的地址计算**：
  - 对于数组 `T A[N]`，元素 `A[i]` 的地址为：`A + i * sizeof(T)`
- **汇编中的数组访问**：使用比例变址寻址方式
  - 示例：`movl (%rdi, %rsi, 4), %eax` → 访问 `A[i]`，其中 `%rdi` 是数组基址，`%rsi` 是索引 `i`

### 3.8.2 指针运算

- **指针加减整数**：`p + i` → `(char *)p + i * sizeof(*p)`
- **指针相减**：`p - q` → `((char *)p - (char *)q) / sizeof(*p)`
- **指针比较**：比较两个指针的地址值

### 3.8.3 嵌套的数组

- **二维数组**：`T A[M][N]`

- 按行优先顺序存储
- 元素 `A[i][j]` 的地址为: `A + (i * N + j) * sizeof(T)`
- 汇编中的二维数组访问:
  - 示例: `movl (%rdi, %rdx, 4), %eax` → 访问 `A[i][j]`, 其中 `%rdi` 是数组基址,  
`%rdx = i * N + j`

### 3.8.4 定长数组

- 定长数组的优化: 编译器可以利用数组长度是常数的信息进行优化
- 示例: 矩阵乘法的优化

### 3.8.5 变长数组

- C99变长数组: 数组长度可以是变量, 必须在运行时确定
- 汇编中的变长数组访问:
  - 数组基址存储在寄存器中
  - 数组长度存储在寄存器中
  - 元素地址计算: `base + i * size`, 其中size是每个元素的大小

## 3.9 异质的数据结构

### 3.9.1 结构

- 结构的内存布局:
  - 成员按声明顺序存储
  - 可能会有填充字节, 以满足对齐要求
- 对齐规则:
  - 每个成员的地址必须是其大小的整数倍
  - 整个结构的大小必须是最大成员大小的整数倍
- 汇编中的结构访问:
  - 结构基址存储在寄存器中
  - 成员通过偏移量访问
  - 示例: `movl 8(%rdi), %eax` → 访问结构中偏移量为8的成员

### 3.9.2 联合

- 联合的内存布局:

- 所有成员共享同一块内存空间
- 联合的大小等于最大成员的大小
- **联合的用途：**
  - 节省内存空间
  - 实现不同数据类型之间的转换

### 3.9.3 数据对齐

- **对齐的原因：**
  - 提高内存访问效率
  - 某些硬件平台要求数据必须对齐，比如x86-64架构
- **对齐原则：**
  - 基本数据类型的地址必须是其大小的整数倍
  - 结构的地址必须是其最大成员大小的整数倍
- **强制对齐：**使用 `__attribute__((aligned(n)))` 属性

## 3.10 理解指针

- **指针的本质：**指针是一个内存地址
- **指针类型：**指针的类型决定了指针解引用时访问的内存大小和解释方式
- **指针与数组：**数组名在大多数情况下会转换为指向数组第一个元素的指针
- **指针与函数：**函数指针指向函数的代码地址
- **空指针：**值为0的指针，表示不指向任何有效的内存地址
- **野指针：**指向无效内存地址的指针，使用野指针会导致未定义行为

## 3.11 内存越界引用和缓冲区溢出

### 3.11.1 缓冲区溢出

- **定义：**向缓冲区写入超过其容量的数据，导致覆盖相邻的内存区域
- **危害：**
  - 破坏程序的运行时栈
  - 覆盖返回地址，使程序跳转到恶意代码
  - 执行任意代码
- **示例：**gets函数的缓冲区溢出漏洞

### 3.11.2 对抗缓冲区溢出攻击

- **栈随机化**：每次程序运行时，栈的起始地址都不同
- **栈保护**：在栈帧中插入金丝雀值，函数返回前检查金丝雀值是否被修改
- **限制可执行代码区域**：将数据段和栈段标记为不可执行
- **地址空间布局随机化 (ASLR)**：将程序的代码段、数据段和栈段加载到随机的地址

## 3.12 浮点代码

### 3.12.1 浮点寄存器和指令

- **SSE浮点单元**：
  - 16个XMM寄存器 (%xmm0-%xmm15)，每个128位
  - 可以存储4个单精度浮点数或2个双精度浮点数
- **AVX扩展**：
  - 16个YMM寄存器 (%ymm0-%ymm15)，每个256位
  - 可以存储8个单精度浮点数或4个双精度浮点数
- **浮点指令后缀**：
  - `ss`：标量单精度
  - `sd`：标量双精度
  - `ps`：打包单精度
  - `pd`：打包双精度

### 3.12.2 浮点数据传送

- **标量传送**：
  - `movss`：传送标量单精度
  - `movsd`：传送标量双精度
- **打包传送**：
  - `movaps`：传送对齐的打包单精度
  - `movapd`：传送对齐的打包双精度
  - `movups`：传送未对齐的打包单精度
  - `movupd`：传送未对齐的打包双精度

### 3.12.3 浮点算术操作

- **标量算术：**
  - `addss`：标量单精度加法
  - `subss`：标量单精度减法
  - `mulss`：标量单精度乘法
  - `divss`：标量单精度除法
  - 双精度版本将 `ss` 改为 `sd`
- **打包算术：**
  - `addps`：打包单精度加法
  - 双精度版本将 `ps` 改为 `pd`

### 3.12.4 浮点比较和条件传送

- **浮点比较：**
  - `ucomiss`：比较标量单精度，设置条件码
  - `ucomisd`：比较标量双精度，设置条件码
- **浮点条件传送：**
  - `cmovss`：标量单精度条件传送
  - `cmovsd`：标量双精度条件传送

### 3.12.5 浮点过程

- **浮点参数传递：**
  - 前8个浮点参数通过XMM寄存器传递：`%xmm0-%xmm7`
  - 超过8个的浮点参数通过栈传递
- **浮点返回值：**通过 `%xmm0` 寄存器传递
- **寄存器使用惯例：**
  - 调用者保存：`%xmm0-%xmm7, %xmm16-%xmm31`
  - 被调用者保存：`%xmm8-%xmm15`

## 第5章 优化程序性能

### 1. 编译器优化的局限

编译器必须保证优化前后行为一致，以下两种情况阻碍优化：

- **内存别名：**两个指针可能指向同一地址，编译器不敢合并操作

```
void twiddle1(long *xp, long *yp) { *xp += *yp; *xp += *yp; } //
xp==yp时*xp变4倍
void twiddle2(long *xp, long *yp) { *xp += 2 * *yp; } //
xp==yp时*xp变3倍
```

- **函数副作用：**编译器无法确定函数是否修改全局状态，不敢移除或重复调用

## 2. 性能度量：CPE

- **CPE (Cycles Per Element)：**每元素周期数，程序运行时间  $\approx K + CPE \times n$
- 用CPE而非"每次循环周期数"：循环展开使每次循环处理多个元素，CPE更准确

## 3. 三种基本优化

| 优化        | 问题         | 方法          | 示例                                   |
|-----------|------------|-------------|--------------------------------------|
| 代码移动      | 循环内重复计算不变量 | 将不变计算移到循环外  | <code>vec_length(v)</code> 提取到循环外    |
| 减少过程调用    | 函数调用开销大    | 内联/提取到循环外   | <code>get_vec_start(v)</code> 提取到循环外 |
| 消除不必要内存引用 | 每次迭代读写内存慢  | 用寄存器累积，最后写回 | 见下方示例                                |

```
// 低效：每次循环通过指针读写内存
for (i = 0; i < len; i++) *dest = *dest OP data[i];
// 优化：寄存器累积，一次写回
float acc = IDENT;
for (i = 0; i < len; i++) acc = acc OP data[i];
*dest = acc;
```

编译器无法确定指针是否别名，不敢自动做此优化

## 4. 现代处理器与性能界限

- 处理器采用**超标量**（每周期发射多条指令）和**乱序执行**技术
- 两个性能下界：
  - **延迟界限**：数据相关迫使操作串行执行时的最小CPE（合并运算 $L \times n$ 的关键路径）
  - **吞吐量界限**：功能单元原始计算能力的上限，是性能的**终极限制**

## 4.1 功能单元性能（Intel Core i7 Haswell）

| 运算 | 整数延迟/发射/容量    | 浮点延迟/发射/容量    |
|----|---------------|---------------|
| 加法 | 1 / 1 / 4     | 3 / 1 / 1     |
| 乘法 | 3 / 1 / 1     | 5 / 1 / 2     |
| 除法 | 330 / 330 / 1 | 315 / 315 / 1 |

- **延迟**：完成运算的总周期数
- **发射时间**：连续同类型运算的最小间隔周期数；发射=1 → **完全流水线化**（每周期可开始新运算）
- **容量**：能执行该运算的功能单元数量；**吞吐量** = 容量 / 发射时间

## 4.2 数据流图与关键路径

- 将循环操作建模为**数据流图**，节点=操作，边=数据依赖
- **关键路径**：从一次迭代到下一次迭代形成的数据相关链，长度决定CPE下界
- 合并运算中 acc 的连续更新形成关键路径， $CPE \geq L$ （合并运算延迟）

# 5. 循环展开与并行优化

## 5.1 $k \times 1$ 循环展开（简单展开）

```
for (i = 0; i < limit; i += 2) {
    acc = acc OP data[i];      // 仍依赖上次迭代的acc
    acc = acc OP data[i+1];    // 仍依赖上一行的acc
}
```

- **效果**：减少循环开销（分支、索引计算），但**不能突破延迟界限**（acc仍串行依赖）
- 整数加法CPE从1.27→1.00，其他运算CPE基本不变

## 5.2 $k \times k$ 循环展开（多累积变量）

```
float acc0 = IDENT, acc1 = IDENT;
for (i = 0; i < limit; i += 2) {
    acc0 = acc0 OP data[i];    // acc0与acc1独立，可并行
    acc1 = acc1 OP data[i+1];
}
*dest = acc0 OP acc1;
```

- **核心：**k个独立累积变量打破数据相关，利用多个功能单元并行
- **效果：**接近吞吐量界限（浮点乘CPE 5.01→0.51，整数加CPE 1.27→0.54）

### 5.3 $k \times 1a$ 循环展开（重新结合变换）

```
// 原始：acc先与data[i]结合，再与data[i+1]结合（两次都依赖acc）
acc = acc OP data[i];
acc = acc OP data[i+1];
// 重新结合：data[i]与data[i+1]先结合，再与acc结合（只有一次依赖acc）
acc = acc OP (data[i] OP data[i+1]);
```

- **核心：**改变结合顺序，关键路径上只有1次对acc的依赖（而非2次）
- **效果：**与 $k \times k$ 类似，可接近吞吐量界限
- **注意：**浮点运算不保证结合律，可能影响精度

### 5.4 三种展开方式对比

| 方式    | 记号            | 关键路径长度         | 能否突破延迟界限 | 典型效果    |
|-------|---------------|----------------|----------|---------|
| 简单展开  | $k \times 1$  | $L \times n$   | ✗        | 仅减少循环开销 |
| 多累积变量 | $k \times k$  | $L \times n/k$ | ✓        | 接近吞吐量界限 |
| 重新结合  | $k \times 1a$ | $L \times n/k$ | ✓        | 接近吞吐量界限 |

## 6. 限制因素

- **寄存器溢出：**k过大时累积变量超过可用寄存器，编译器将变量存到栈上，优势消失。x86-64寄存器足够多，多数循环在溢出前已达吞吐量界限
- **分支预测错误处罚：**约15~19个时钟周期。可预测的分支（如循环结束）不必担心；不可预测分支优先用条件传送（cmov）



# 7. 性能提高技术总结

| 级别   | 技术                                                     | 核心思想             |
|------|--------------------------------------------------------|------------------|
| 高级设计 | 合适的算法与数据结构                                             | 避免渐进低效           |
| 基本编码 | 代码移动、减少调用、消除多余内存引用                                     | 消除明显低效           |
| 低级优化 | 循环展开、多累积变量、重新结合变换                                      | 利用指令级并行          |
| 程序剖析 | GPROF定位瓶颈（ <code>gcc -pg</code> → <code>gprof</code> ） | 指导优化方向（Amdahl定律） |

## 第7章 链接

### 1、链接的基本概念与作用

- **定义：**链接是将各种代码和数据片段收集并组合成一个单一可执行文件的过程，这个文件可以被加载到内存中执行
- **执行时机：**
  - 编译时：源代码被翻译成机器代码时
  - 加载时：程序被加载器加载到内存时
  - 运行时：由应用程序在执行过程中调用
- **核心价值：**
  - 实现了代码的模块化开发，允许程序被分割成多个源文件
  - 支持库的使用，实现代码复用
  - 分离了编译和链接，修改单个文件无需重新编译整个程序

### 2、目标文件格式（ELF）

- **可重定位目标文件（.o）：**包含二进制代码和数据，可与其他可重定位目标文件合并创建可执行文件
- **可执行目标文件：**包含可直接加载到内存执行的代码和数据
- **共享目标文件（.so）：**特殊的可重定位目标文件，可在加载时或运行时被动态链接
- **ELF文件结构：**
  - ELF头：描述文件整体结构，包括入口点地址、节头表偏移等
  - 节头表：描述各个节的位置和大小

- 关键节：.text（代码）、.data（已初始化数据）、.bss（未初始化数据）、.symtab（符号表）、.rel.text（重定位信息）、.rel.data（重定位信息）、.strtab（字符串表）

## 3、符号与符号表

- **符号类型：**
  - 全局符号：在本模块定义，可被其他模块引用（如非static函数和全局变量）
  - 外部符号：在其他模块定义，本模块引用（用extern声明）
  - 本地符号：仅在本模块内部使用（static函数和static变量）
- **符号解析：**链接器将每个符号引用与唯一的符号定义关联起来
- **强符号与弱符号：**
  - 强符号：函数和已初始化的全局变量
  - 弱符号：未初始化的全局变量
  - 规则：不能有多个同名强符号；一个强符号和多个弱符号同名时，选择强符号；多个弱符号同名时，任选一个

## 4、重定位

- **目的：**将符号引用与符号定义的内存地址关联起来
- **两个步骤：**
  1. **重定位节和符号定义：**将所有相同类型的节合并，为每个节和符号分配运行时内存地址
  2. **重定位节中的符号引用：**修改代码和数据节中所有对符号的引用，使其指向正确的运行时地址
- **重定位条目：**描述如何修改一个引用，包含偏移量、符号索引和类型
- **常见重定位类型：**
  - ELF定义了32种不同的重定位类型，书中介绍了两种最常用的类型：
  - R\_X86\_64\_PC32：相对PC的32位重定位
  - R\_X86\_64\_32：绝对32位重定位

## 5、静态链接

- **过程：**将多个可重定位目标文件和静态库（.a）合并成一个可执行文件
- **静态库：**由多个可重定位目标文件打包而成的归档文件
- **链接器算法：**
  - 按命令行顺序扫描目标文件和库

- 维护一个未解析符号集合
- 遇到目标文件时，将其符号加入符号表，解析未解析符号
- 遇到库时，仅提取能解析当前未解析符号的目标文件
- **注意事项：**库通常放在命令行末尾，否则可能导致符号未定义错误

## 6、动态链接

- **基本思想：**将库的链接推迟到加载或运行时
- **共享库 (.so)：**
  - 所有进程共享同一份库代码在内存中的副本
  - 库代码是位置无关的 (PIC)，可加载到内存任意位置
- **加载时动态链接：**程序加载时由动态链接器完成链接
- **运行时动态链接：**程序运行过程中通过dlopen()等函数显式加载和链接库
- **优势：**
  - 节省内存和磁盘空间
  - 库更新无需重新链接程序
  - 支持多语言程序开发

## 7、位置无关代码 (PIC)

- **定义：**代码中不含绝对地址，无需修改即可加载到内存任意位置执行
- **核心机制：**内部引用用RIP相对寻址，外部引用通过GOT/PLT间接访问

### 7.1 延迟绑定 (Lazy Binding)

- **核心思想：**函数地址不在加载时解析，而在**第一次调用时**才解析（避免启动时解析数百个未使用的库函数）
- **PLT (过程链接表)：**每项16字节，含跳转指令和解析桩代码
- **GOT (全局偏移表)：**每项8字节，存放函数/变量的实际地址；初始值指向PLT[0]，解析后更新为真实地址

调用流程（以printf为例）：

首次调用：`call printf@PLT → PLT[printf]: jmp *GOT[printf]`  
→ `GOT[printf]`初始指向`PLT[0]` → `PLT[0]: push GOT[1], jmp *GOT[2]`  
→ 动态链接器(`_dl_runtime_resolve`)查找`printf`地址，写入`GOT[printf]`，跳转执行

后续调用：`call printf@PLT → PLT[printf]: jmp *GOT[printf]` → 直接跳转到`printf`（一步到位）

## 7.2 PIC访问全局变量

| 场景     | 方式      | 是否经过GOT | 示例                                                                 |
|--------|---------|---------|--------------------------------------------------------------------|
| 模块内部变量 | RIP相对寻址 | 否       | <code>movq my_var(%rip), %rax</code>                               |
| 外部全局变量 | GOT间接寻址 | 是       | <code>movq ext_var@GOTPCREL(%rip), %rax → movq (%rax), %rax</code> |

外部变量必须通过GOT：共享库加载地址不确定，编译时无法知道绝对地址，GOT由动态链接器在加载时填充

## 7.3 PIC库函数调用

| 场景     | 方式              | 是否经过PLT/GOT | 示例                              |
|--------|-----------------|-------------|---------------------------------|
| 模块内部函数 | RIP相对寻址         | 否           | <code>call my_func(%rip)</code> |
| 外部库函数  | PLT + GOT（延迟绑定） | 是           | <code>call printf@PLT</code>    |

## 7.4 总结

| 机制        | 解决的问题       | 实现方式                    |
|-----------|-------------|-------------------------|
| 延迟绑定      | 避免启动时解析所有函数 | PLT + GOT，首次调用时解析       |
| PIC访问全局变量 | 代码不依赖绝对地址   | 内部RIP相对寻址，外部GOT间接访问     |
| PIC调用库函数  | 代码不依赖绝对地址   | 内部RIP相对寻址，外部PLT+GOT延迟绑定 |

## 8、常见链接问题与工具

- 常见错误：

- 符号未定义（undefined reference）：**引用**了符号但没找到**定义**。
- 多重定义（multiple definition）：在多个模块中定义了相同**强符号**。
- 库版本不兼容

- 常用工具：

- readelf：查看ELF文件结构
- objdump：反汇编目标文件
- nm：列出符号表
- ldd：查看可执行文件依赖的共享库
- strip：去除符号表和调试信息